Design and Verification of a BRAM-Backed SRAM Processing-in-Memory Approximation for INT8 Matrix–Vector Products

Rahul Nerella, Uttiyo Majumder, and Kevin Velanjickal
Department of Electronics and Communication Engineering
Vellore Institute of Technology, Vellore, India
V-SPACE FPGA Build Challenge 2026

***Abstract*—This paper presents a parameterized, synthesizable SystemVerilog engine that emulates the digital dataflow of an SRAM processing-in-memory (PIM) block using FPGA-friendly synchronous block RAM (BRAM). The engine computes signed INT8 row-wise dot products, $y_r = \sum_{c=0}^{C-1} w_{r,c}a_c$, using a single multiply–accumulate datapath, a synchronous weight-memory read, a registered activation vector, an accumulator, and a result memory. The submitted testbench is configured for a 4-row by 8-column case and checks a known-answer pattern, 30 randomized patterns, arithmetic corner cases, reset during computation, back-to-back transactions, and the synchronous result-read protocol. The most recent archived submission-tree rerun records P1–P5 as passing with no compile errors or warnings. A legacy transcript from an earlier project tree records a pass for the Python golden-vector phase, but the newer rerun skipped P6; therefore, the current revision's P6 result is reported as not freshly verified. Model-derived figures report 36 cycles (360 ns at 100 MHz) for the 4×8 workload and a maximum observed absolute accumulator value of 100,843 across 2,000 16×16 vectors. These estimates are not synthesis or hardware measurements. The implementation is explicitly a digital SRAM-PIM approximation and does not reproduce analog bitline computation, ADC behavior, device variation, or silicon energy.**

***Index Terms*—FPGA, processing-in-memory, SRAM, BRAM, matrix–vector multiplication, INT8, multiply–accumulate, SystemVerilog, RTL verification.**

I. Introduction

Moving weights and activations between memory and arithmetic units can dominate the cost of inference workloads. Compute-in-memory (CiM) architectures seek to reduce this movement by performing some computation within or close to a memory array. A conventional FPGA does not expose an analog SRAM-CiM bitcell macro, so an FPGA project can validate only selected digital interfaces and sequencing unless a dedicated memory macro or analog model is available.

This experiment builds a transparent digital approximation of a memory-centric dot-product engine. Signed 8-bit weights are held in a synchronous 1-read/1-write memory intended to infer block RAM; signed activations reside in registers; a shared multiply–accumulate unit streams one weight per cycle; and completed row sums are written to a result register array. The design supports loading, start/busy/done control, reset recovery, and a synchronous result-read port. The goal is functional correctness and a reusable digital dataflow proxy for the more integrated SRAM-CIM experiment, not a claim of analog CiM operation.

II. Architecture and Design

A. Computational Model

For row r, the design computes

$$y_r = \sum_{c=0}^{C-1} w_{r,c}a_c, \qquad w_{r,c}, a_c \in [-128, 127]. \tag{1}$$

The RTL stores weights and activations as signed `DATA_W`-bit elements and stores results in `ACC_W` bits. The default parameters are $R = 16$, $C = 16$, `DATA_W` $= 8$, and `ACC_W` $= 32$. The delivered self-checking testbench overrides the dimensions to $R = 4$ and $C = 8$, so the primary functional rerun covers 32 products per vector and four output rows. Accumulation uses signed two's-complement arithmetic in the configured accumulator width.

B. Memory Organization and Datapath

The weight memory $wmem$ has RC locations and a one-cycle synchronous read interface; writes are accepted through a dedicated load port when the engine is not computing. Activation memory $amem$ has C signed elements. The result array $rmem$ stores one accumulator-width output per row and exposes a synchronous read port with `r_rd_valid`. The primary datapath uses a single signed multiplication and one accumulator feedback path, so the reduction is serialized rather than parallelized across all columns.

The RTL separates memory-read issue from consumption to account for the synchronous weight-memory latency. An issue counter presents the next flattened row-major weight address and an associated column index; a delayed valid bit identifies when that weight can be consumed. At each row boundary, the accumulated value is committed to the corresponding result entry and the accumulator is cleared for the following row. The issue counter is wider than the weight address so it can represent the terminal count RC without wrapping before completion.

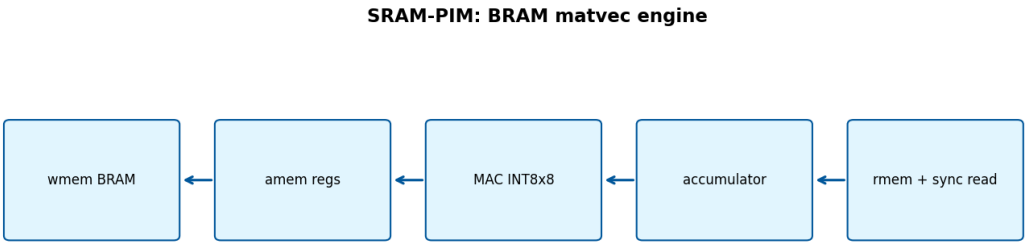


Fig. 1. Architecture diagram supplied with the project. The RTL implements the memory/accumulator control as a synchronous digital matvec approximation; the figure is not a transistor-level SRAM-CiM schematic.

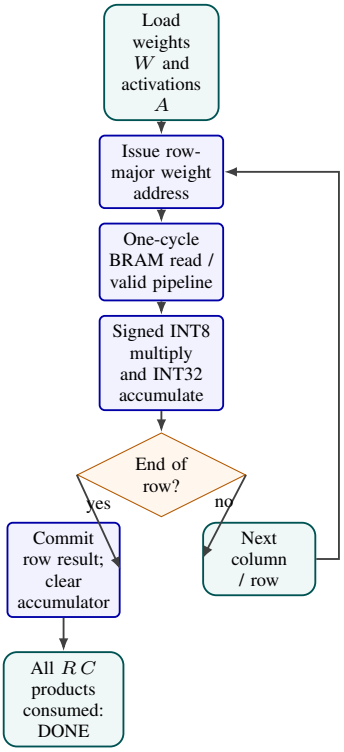


Fig. 2. Conceptual compute sequencing. The RTL delays consumption to match the synchronous weight-memory read and commits a result at each row boundary.

C. Finite-State Control and Interface

The controller uses `S_IDLE`, `S_COMP`, and `S_DONE`. A start pulse accepted while idle or done initializes the issue/consume counters and accumulator. During computation, writes to weight and activation memories are blocked. The done signal remains asserted while the controller stays in the DONE state and is cleared as a new computation is accepted or reset is asserted. Reset is active-low and asynchronously clears control and pipeline-valid state. Result reads have one-cycle validity timing.

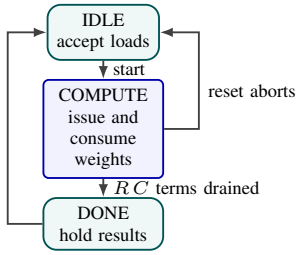


Fig. 3. Three-state controller at the interface level. The testbench explicitly verifies reset recovery and repeated transactions.

TABLE I
FUNCTIONAL TEST PHASES AND AVAILABLE EVIDENCE

ID	Test intent	Evidence
P1	Known answer; expected rows 0 and 1 are 92 and 60	Pass
P2	30 randomized signed weight/activation patterns	Pass
P3	Zero, maximum, minimum, and mixed-sign corners	Pass
P4	Reset mid-compute; check busy/done clear and recover	Pass
P5	Back-to-back runs, read-valid latency, stray start while busy	Pass
P6	Cross-check packaged Python golden vectors	Pending

III. Verification Methodology

The testbench `pim_mac_tb.sv` maintains independent shadow arrays W and A and computes an expected row using a reference function that loops over the column dimension. The self-checking phases are listed in Table I; the testbench uses standard SystemVerilog constructs and project-local source files [3], [4]. The fixed known-answer test uses $W_{r,c} = rC + c - 16$ and $A_c = c - 4$; the supplied comments state expected row sums of 92 and 60 for rows 0 and 1. Random tests regenerate signed operands and compare each result row. Corner cases include zero operands, all $+127$, all -128, and -128×127, targeting signed multiplication and accumulation errors.

The most recent archived submission-tree rerun, dated 2026-10-10, records P1–P5 as passing, with zero compile errors and zero warnings. Its P6 phase was skipped because the older invocation did not stage vectors at the legacy path. The current `Simulation/run_sim.do` explicitly copies

TABLE II
REPORTED CONFIGURATION AND MODEL-DERIVED METRICS

Metric	Value
RTL test configuration	4×8
Default RTL dimensions	16×16
Operand / accumulator width	INT8 / INT32
4×8 latency at 100 MHz	36 cycles / 360 ns
4×8 vector-rate estimate	2.78 M vectors/s
16×16 vector-rate estimate	0.385 M vectors/s
Headroom trials / observed maximum	2000 / 100843
Recent testbench status	P1–P5 pass; P6 pending

`Testbench/vectors` to `tb/vectors` before simulation, and the newer testbench raises an error if those files remain unavailable. A September transcript from an earlier project tree reports a P6 pass, but that historical run does not independently establish P6 correctness for the current submission revision. ModelSim was unavailable during this report build, so a fresh P1–P6 run could not be executed here.

IV. Results and Discussion

A. Known-Answer and Model Validation

The project result graphic shows the four known-answer row sums as 92, 60, 28, and -4. The red marker on row 2 includes an annotation in the graphic explaining that an earlier comment said 20 while the actual result for the present pattern is 28; the testbench's current independent reference calculation is used as the authority. The same figure includes model validation checks V1–V5 at 100%. These checks originate from the separate model/validation workflow and must not be interpreted as coverage of five distinct RTL or CNN layers in the submitted testbench.

B. Latency and Throughput Estimates

The supplied model summary records a measured 36-cycle start-to-done latency for a 4×8 vector, corresponding to 360 ns at 100 MHz. This agrees with the model $L = RC + 4$ for the tested shape. The corresponding reciprocal-rate estimate is approximately $100\ \text{MHz}/36 = 2.78$ million vectors/s. For a 16×16 vector, $L = 256 + 4 = 260$ cycles gives approximately 384,615 vectors/s. The model assumes one multiply–accumulate operation per cycle and excludes host communication, loading overhead, and physical implementation effects.

C. Accumulator Headroom

The accompanying model CSV reports 2,000 random 16×16 INT8 vectors and a maximum observed absolute accumulator magnitude of 100,843. Relative to the maximum positive signed INT32 value, the plotted magnitude is approximately 0.0047%, rounded to 0.005% in the source plot. This supports comfortable numeric headroom for that sampled workload, but is not an exhaustive proof over every possible input matrix; a simple worst-case bound is $C \times 128 \times 128$, which is 262,144 for $C = 16$ and remains well within signed INT32 range.

V. FPGA Implementation Considerations and Limitations

The repository includes a Nexys A7 XDC template, but pin constraints are not evidence of a completed FPGA build. No Vivado synthesis, BRAM-inference report, timing report, resource utilization report, or board-level demonstration is included for the current experiment. The synchronous read and single-clock memory style are written to encourage BRAM inference, but this needs confirmation in the synthesis reports and follows standard FPGA memory-inference guidance [1], [2].

The experiment does not model SRAM bitline accumulation, analog conductance, DAC/ADC conversion, process variation, read noise, or memory-cell reliability. As a result, it cannot support claims about silicon CiM accuracy, energy per operation, or power savings. The current architecture is also serial: it performs one product accumulation per cycle, and loading the weight matrix and activation vector is outside the latency estimate. The results are appropriate for demonstrating digital dataflow and control behavior, not for comparing analog CiM energy with conventional digital accelerators.

VI. Applications and Future Work

Potential uses include validating memory-centric accelerator protocols, teaching synchronous memory latency, verifying dot-product control sequences, and serving as a functional reference for the more integrated RISC-V SRAM-CIM experiment. Next steps are to run the current P1–P6 testbench under ModelSim, archive a fresh transcript and waveform, confirm BRAM inference and timing in Vivado, measure resource utilization, and execute a documented board test. Further architectural work could replicate MAC lanes or use a reduction tree to improve throughput, while preserving the load/start/result interface.

VII. Conclusion

A BRAM-backed SystemVerilog matvec engine was documented and evaluated as a digital approximation of SRAM processing-in-memory. The recent archived rerun supports the known-answer, randomized, arithmetic-corner, reset-recovery, and transaction-protocol tests (P1–P5). The packaged Python golden-vector phase P6 remains pending fresh verification on the current project revision, despite a successful older transcript. Model plots give a 36-cycle estimate for a 4×8 vector and show INT32 accumulator headroom for the sampled 16×16 workload. Neither physical FPGA implementation nor analog SRAM-CiM behavior has been demonstrated.

References

[1] AMD Xilinx, "Vivado Design Suite User Guide: Synthesis (UG901)," documentation for inferring FPGA memories and sequential logic. Available: https://docs.amd.com/r/en-US/ug901-vivado-synthesis.

[2] Intel, "Quartus Prime User Guide: Design Recommendations," FPGA memory coding and implementation guidance. Available: https://www.intel.com/content/www/us/en/docs/programmable/683082/current/introduction.html.

[3] IEEE, "IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language," IEEE Std 1800.

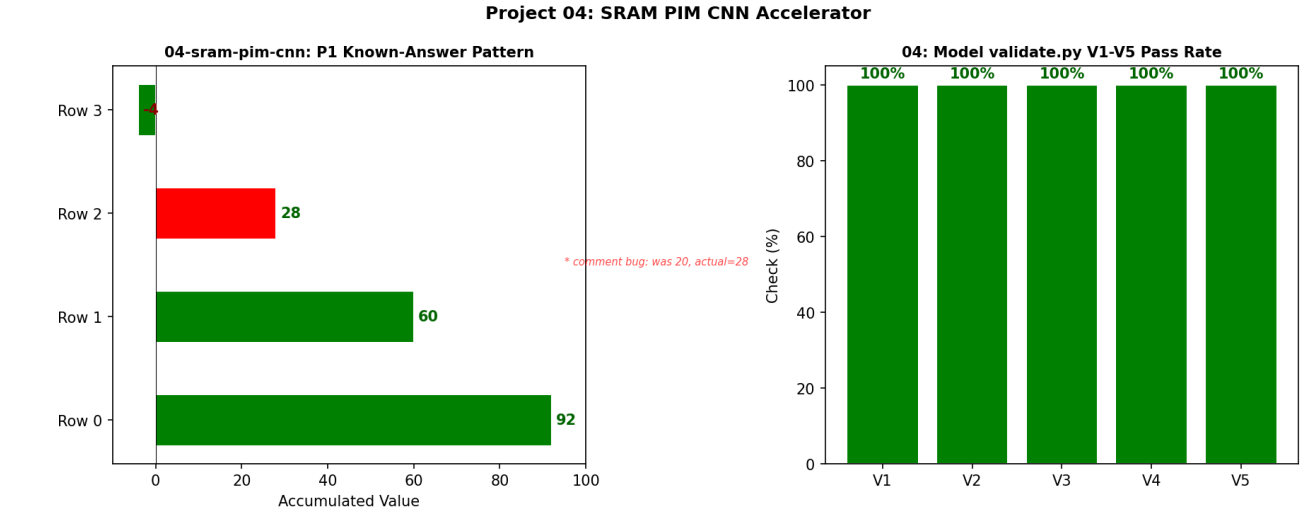


Fig. 4. Project-provided PIM summary graphic. The left panel reports the four known-answer outputs; the right panel reports V1–V5 model-check pass rates. These are not FPGA hardware measurements, and the model checks are distinct from the RTL test phases summarized in Table I.

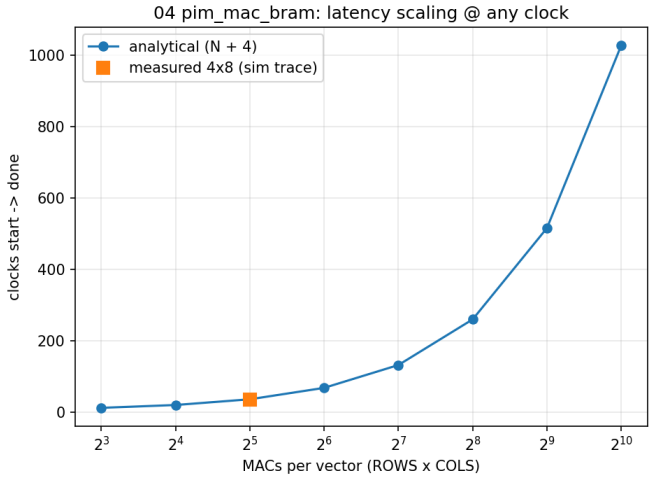


Fig. 5. Archived latency scaling plot. The orange marker denotes the 4×8 simulation trace; the remaining curve is analytical, not a sweep of measured hardware.

[4] Project artifacts: `pim_mac_bram.sv`, `pim_mac_tb.sv`, the 2026-10-10 rerun log, and `results_summary.csv`, V-SPACE FPGA Build Challenge 2026 repository.

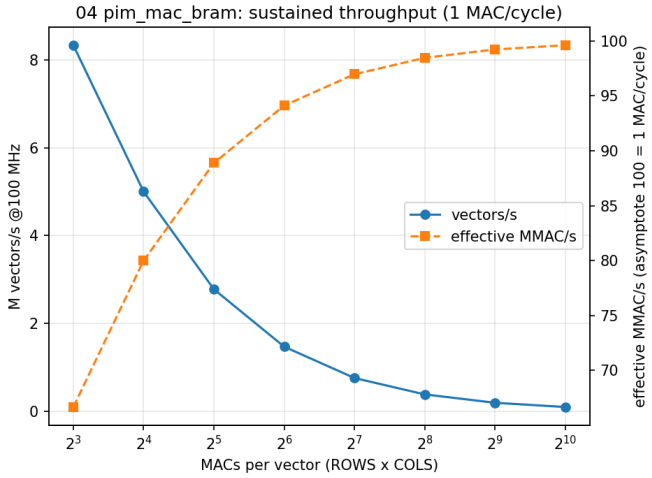


Fig. 6. Analytical sustained-throughput estimate for a one-MAC-per-cycle design at 100 MHz. Effective MMAC/s approaches 100 as vector size increases because fixed transaction overhead is amortized.

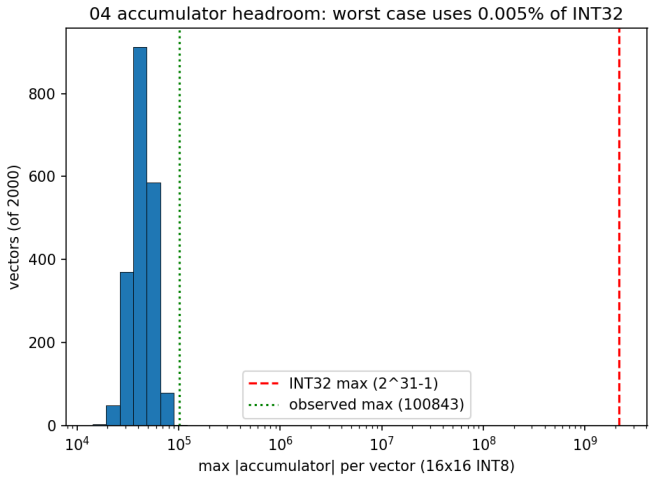


Fig. 7. Model-derived distribution of maximum absolute accumulators across 2,000 vectors. The histogram is statistical evidence for the sampled dataset, not an RTL waveform or physical measurement.